

Inteligencia Artificial Generativa Avanzada y Sistemas Multi-Agente

Apuntes de clase — UTDT
Prof. Francisco Traversaro

Índice

1. De vectores a un LLM — Fundamentos matemáticos	3
1.1. El camino hacia un LLM	3
1.2. Vectores, producto interno y embeddings	3
1.3. Aplicaciones lineales y matrices	3
1.4. Unidad afín + no linealidad	3
1.5. Aprendizaje supervisado como optimización	4
1.6. Gradiente y descenso por gradiente	4
1.7. Backpropagation como diferenciación automática	4
1.8. Notebook: Motor de autograd (micrograd)	4
2. Attention is All You Need — Transformers	5
2.1. El punto de partida	5
2.2. Del texto a tokens	5
2.3. El bigrama: el modelo más simple	5
2.4. Promediar el pasado: darle memoria	5
2.5. Self-Attention	6
2.6. Multi-Head Attention	6
2.7. El bloque Transformer completo	6
2.8. Encoder vs Decoder	7
3. Detrás de la IA — Tokenizers	7
3.1. Por qué importa la tokenización	7
3.2. De texto a bytes	7
3.3. Byte-Pair Encoding (BPE)	7
3.4. Encode y Decode	8
3.5. Regex splitting (GPT-2/GPT-4)	8
3.6. Tokens especiales	8
3.7. Tokenizers reales y algoritmos	8
3.8. Notebook: Tokenización	9
3.9. Notebook: “¿Qué es realmente la IA?”	9
4. Embeddings	10
4.1. Parte 1: Word Embeddings	10
4.1.1. Los embeddings de un LLM se crean dentro del modelo	10
4.1.2. Similitud coseno	10
4.1.3. One-hot: identidad, cero significado	10

4.1.4.	Hipótesis distribucional y conteo	10
4.1.5.	Word2Vec: aprender prediciendo	10
4.1.6.	GloVe: contar y después aprender	11
4.1.7.	Techo: static embeddings y polisemia	11
4.1.8.	Sesgos	11
4.2.	Parte 2: Sentence Embeddings	11
4.2.1.	El problema	11
4.2.2.	Baseline: promediar word vectors	11
4.2.3.	BERT: el encoder	12
4.2.4.	SBERT (Sentence-BERT)	12
4.2.5.	Embedders basados en decoder (GPT)	12
4.2.6.	Tres “embeddings” distintos	12
4.2.7.	Búsqueda semántica = la “R” de RAG	12
4.2.8.	Los tres modelos de un RAG	13
4.2.9.	Qué embedder usar (2026)	13
4.2.10.	Lo que falta para RAG en serio (Semana 8)	13
4.3.	Notebook: Word Embeddings (N1)	13
4.4.	Notebook: Sentence Embeddings (N2)	13
5.	Pretraining y Fine-tuning	14
5.1.	Parte 1: Preentrenamiento — De la web cruda al modelo base	14
5.1.1.	Tres etapas: masa, silueta, cara	14
5.1.2.	Los datos: curación empírica	14
5.1.3.	La escala: Scaling Laws	15
5.1.4.	El modelo base	15
5.1.5.	Continued pretraining vs fine-tuning	15
5.2.	Parte 2: Fine-tuning eficiente	15
5.2.1.	Instruction tuning (SFT)	15
5.2.2.	PEFT: Parameter-Efficient Fine-Tuning	16
5.2.3.	LoRA (Low-Rank Adaptation)	16
5.2.4.	QLoRA	16
5.2.5.	Síntesis	17
5.3.	Notebook: Fine-tuning eficiente	17
6.	Alineamiento y Evaluación	17
6.1.	Parte 1: Alineamiento	17
6.1.1.	La analogía de Karpathy	17
6.1.2.	Dominios verificables: el loop de RL	18
6.1.3.	Dominios no verificables: preferencias	18
6.1.4.	Reward Model	18
6.1.5.	RLHF	18
6.1.6.	DPO (Direct Preference Optimization)	18
6.1.7.	Constitutional AI (RLAIF)	19
6.1.8.	Síntesis del alineamiento	19
6.2.	Parte 2: Evaluación	19
6.2.1.	Benchmarks	19
6.2.2.	Evaluación por preferencia	19
6.2.3.	Modos de falla	20
6.2.4.	Responsible AI	20
6.2.5.	Síntesis de evaluación	20
	Lecturas recomendadas	21

Clase 1: De vectores a un LLM — Fundamentos matemáticos

1.1 El camino hacia un LLM

Todo LLM se apoya en una cadena de bloques fundamentales:

Vectores/embeddings $\rightarrow$ **Red neuronal** $\rightarrow$ **Atención** $\rightarrow$ **Transformer** $\rightarrow$ **LLM**

Los símbolos clave de la clase: $\mathbb{R}^n$ (espacios vectoriales), $\mathbf{w}^\top \mathbf{x}$ (producto interno), $W\mathbf{x} + \mathbf{b}$ (capa afín), ϕ (activación), $L(\theta)$ (pérdida), ∇L (gradiente), AD (diferenciación automática).

1.2 Vectores, producto interno y embeddings

- Un vector $\mathbf{x} = (x_1, \dots, x_n) \in \mathbb{R}^n$ tiene dos lecturas: n -tupla de coordenadas (álgebra) y punto/dirección con norma (geometría).
- En deep learning, todo dato (token, imagen) se codifica como vector en dimensión alta: **representación distribuida**.
- **Embeddings**: vectores aprendidos por palabra. La geometría del espacio captura significado:

$$\text{vec}(\text{rey}) - \text{vec}(\text{hombre}) + \text{vec}(\text{mujer}) \approx \text{vec}(\text{reina})$$

- **Producto interno**: $\mathbf{w}^\top \mathbf{x} = \sum_i w_i x_i = \|\mathbf{w}\| \|\mathbf{x}\| \cos \theta$. Mide alineación/proyección entre vectores. Base de la **similitud coseno**.
- En atención, el peso de un token sobre otro es el producto interno de sus vectores query y key (escalado por $\sqrt{d}$, normalizado con softmax).

1.3 Aplicaciones lineales y matrices

- $W \in \mathbb{R}^{m \times n}$ representa $\mathbb{R}^n \rightarrow \mathbb{R}^m$. Cada componente de la salida es un producto interno de una fila con la entrada.
- Una **capa lineal** son m productos internos en paralelo.
- Componer capas lineales = multiplicar matrices. El producto matriz-matriz domina el costo del entrenamiento $\Rightarrow$ hardware paralelo (GPU).
- **SVD** ($W = U\Sigma V^\top$): toda aplicación lineal es rotación $\cdot$ escala $\cdot$ rotación. Relevante para LoRA y atención de rango bajo.

1.4 Unidad afín + no linealidad

- Unidad: $a = \phi(\mathbf{w}^\top \mathbf{x} + b)$. Capa: $\mathbf{h} = \phi(W\mathbf{x} + \mathbf{b})$.
- **Sin ϕ , apilar capas no agrega expresividad**: la composición de afines es afín. La no linealidad rompe eso.
- Activaciones comunes: ReLU, sigmoide (σ), tanh.
- **Teorema de aproximación universal**: una red con una capa oculta y no linealidad no polinomial aproxima cualquier función continua sobre un compacto (Cybenko 1989, Hornik 1991). No dice cuántas neuronas ni si el entrenamiento las encuentra.

1.5 Aprendizaje supervisado como optimización

- Datos (x_i, y_i) , modelo f_θ , pérdida $L(\theta) = \frac{1}{N} \sum_i \ell(f_\theta(x_i), y_i)$.
- Entrenar = resolver $\min_\theta L(\theta)$ (en general no convexo).
- **Funciones de pérdida:**
 - Regresión: MSE $\ell(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|^2$ (modelo gaussiano).
 - Clasificación: entropía cruzada $\ell = -\sum_i y_i \log \hat{y}_i$ con $\hat{y} = \text{softmax}(z)$ (modelo categórico).
- Elegir una pérdida es, en el fondo, elegir un modelo probabilístico de los datos.

1.6 Gradiente y descenso por gradiente

- $\nabla_\theta L \in \mathbb{R}^p$: vector de derivadas parciales, apunta a la dirección de máximo ascenso.
- Regla de actualización: $\theta \leftarrow \theta - \eta \nabla_\theta L(\theta)$.
- η grande $\rightarrow$ diverge; η chico $\rightarrow$ converge lento.
- **SGD**: gradiente estimado sobre un minibatch (insesgado).
- **Momentum**: acumula inercia. **Adam**: momentum + escala adaptativa por coordenada. Optimizador por defecto en la práctica.

1.7 Backpropagation como diferenciación automática

- **Regla de la cadena**: $\frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \cdot \frac{\partial y}{\partial x}$.
- Backprop = evaluar la cadena de la salida hacia la entrada, reutilizando resultados (producto de Jacobianos).
- **Modo reverso**: 1 escalar de salida (la pérdida), p parámetros $\Rightarrow$ gradiente completo en UNA pasada hacia atrás, costo $O(\text{forward})$. El modo directo costaría $O(p)$.
- **Grafo de cómputo**: nodos = valores intermedios; aristas = operaciones. Forward: evaluar en orden topológico. Backward: recorrer en orden inverso.

1.8 Notebook: Motor de autograd (micrograd)

El laboratorio construye un motor de diferenciación automática en modo reverso en ~ 100 líneas de Python:

1. **Value**: nodo del grafo. Guarda `data`, `grad` ($\partial L / \partial \text{self}$), `_prev` (padres), `_backward` (closure que propaga gradiente).
2. Operaciones (`__add__`, `__mul__`, `tanh`, `exp`): cada una crea un nodo, registra padres, y define su `_backward` con la derivada local.
3. **backward()**: orden topológico + barrido inverso disparando cada `_backward`.
4. **Gradient check**: verificar analítico $\approx$ numérico con diferencias finitas centradas. Regla de oro: *nunca confíes en un gradiente sin verificarlo*.
5. **Neuron** $\rightarrow$ **Layer** $\rightarrow$ **MLP**: todo se apoya en Value. El loop de entrenamiento (forward $\rightarrow$ loss $\rightarrow$ zero_grad $\rightarrow$ backward $\rightarrow$ update) es conceptualmente el mismo con que se entrena un LLM.

Punto clave: los gradientes se **acumulan** (`+=`), no se reemplazan. Por eso `zero_grad()` es imprescindible en cada paso.

Clase 2: Attention is All You Need — Transformers

2.1 El punto de partida

Paper: Vaswani et al., NeurIPS 2017. Introdujo el Transformer: arquitectura basada enteramente en atención, sin recurrencia ni convoluciones. Base de los modelos de lenguaje actuales.

2.2 Del texto a tokens

- Tres granularidades: carácter, subpalabra, palabra.
- V = tamaño del vocabulario (fijo). T = longitud de la secuencia (varía por texto).
- Trade-off: carácter = vocab chico pero secuencias largas; palabra = vocab enorme pero secuencias cortas. Subpalabra (BPE) busca el equilibrio.

2.3 El bigrama: el modelo más simple

- Predice el próximo carácter mirando únicamente el actual. Sin contexto, sin historia.
- `token_embedding_table = nn.Embedding(V, V)`: la fila i son los logits del próximo token dado el token i .
- De logits a probabilidades: `softmax(filac)`.
- Loss antes de entrenar: tabla random $\rightarrow$ predicciones picudas y equivocadas $\rightarrow$ loss $> \ln(V)$ (peor que el “ignorante honesto” uniforme).
- Entrenar un bigrama = contar pares. El gradiente en la fila: `grad = p - onehot(correcto)`.
- **Techo:** no conoce el contexto. Solo codifica “qué caracteres suelen seguir a c ” promediado sobre todo el corpus.

2.4 Promediar el pasado: darle memoria

- Representación de cada token = promedio de todos los tokens hasta él (bag of words del pasado). Ojo causal: solo hacia atrás.
- Tres versiones equivalentes:
 1. For-loop (correcto pero lento).
 2. Matriz triangular normalizada: `wei = tril/sum`, `xbow = wei @ x`.
 3. Softmax de afinidades: `zeros` $\rightarrow$ máscara $-\infty$ $\rightarrow$ softmax $\rightarrow$ `wei @ v`.
- **El molde que no cambia:** (1) armar `wei`, (2) máscara $-\infty$, (3) softmax por fila, (4) `salida = wei @ v`.
- **Problema del promedio:** descarta el **orden** y la **relevancia** (todos pesan igual).

2.5 Self-Attention

- Cambiamos UNA pieza del molde: w_i pasa a calcularse a partir de los tokens (ya no son ceros).
- Tres proyecciones lineales por token:
 - **Query** $q = x \cdot W_Q$: “qué estoy buscando”.
 - **Key** $k = x \cdot W_K$: “qué ofrezco para ser encontrado”.
 - **Value** $v = x \cdot W_V$: “qué apporto si me eligen”.
- Afinidad: $\text{wei}[i, j] = q_i \cdot k_j$ (producto interno).
- Fórmula de una cabeza:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \text{máscara}\right) \cdot V$$

- Escalado $1/\sqrt{d_k}$: control de varianza. Sin él, el softmax se satura y el gradiente se apaga.
- **Positional encoding**: la atención es ciega al orden $\rightarrow$ entrada = embedding(token) + PE(pos).
- “Self”: Q, K, V salen de la misma secuencia. Si K, V vinieran de otra, sería cross-attention.

2.6 Multi-Head Attention

- Una cabeza = un solo par W_Q/W_K = una sola noción de “relevante”. Insuficiente: en lenguaje conviven varios tipos de relación (sintaxis, proximidad, coreferencia).
- Multi-head: H cabezas independientes, cada una con sus propias W_Q, W_K, W_V , en subespacios de tamaño C/H .
- Concatenar + proyección W_O : mezcla los bloques. Entra (B, T, C) , sale (B, T, C) .
- **Costo**: H cabezas de C/H tienen los mismos parámetros que 1 cabeza de C . Expresividad prácticamente gratis.

2.7 El bloque Transformer completo

- **FFN** (Feed-Forward Network): $\text{FFN}(x) = W_2 \cdot \phi(W_1 x + b_1) + b_2$. Cómputo por token (vertical), independiente por posición. Dimensión interna típica $4C$.
- Atención **comunica** tokens; FFN **procesa** cada token.
- **Conexiones residuales**: $x \leftarrow x + \text{Sublayer}(x)$. Camino directo para el gradiente $\rightarrow$ combate el desvanecimiento.
- **LayerNorm**: normaliza cada vector (media 0, varianza 1) sobre sus C componentes. Estabiliza activaciones.
- Bloque completo:

$$\begin{aligned}x &\leftarrow x + \text{MultiHead}(\text{LN}(x)) \\x &\leftarrow x + \text{FFN}(\text{LN}(x))\end{aligned}$$

- Se apilan N bloques. Capas bajas $\rightarrow$ estructura local; altas $\rightarrow$ relaciones abstractas. BERT: $N = 12$, GPT-3: $N = 96$.

2.8 Encoder vs Decoder

Encoder-only	Decoder-only	Encoder-decoder
BERT	GPT (lo que construimos)	T5, paper original
Comprensión	Generación	Secuencia a secuencia

- Encoder: atención bidireccional (sin máscara). Decoder: atención causal (con máscara).
- Cross-attention: Q del decoder, K y V del encoder.
- Se enmascara solo cuando ver un token sería hacer trampa respecto de lo que falta generar.

Clase 3: Detrás de la IA — Tokenizers

3.1 Por qué importa la tokenización

La tokenización está en el origen de muchos comportamientos anómalos de los LLMs. La unidad mínima es el **token**, no el carácter.

Comportamientos que explica:

- **Deletreo:** no deletrea ni cuenta letras dentro de un token.
- **Aritmética:** números segmentados de formas inconsistentes.
- **Otros idiomas:** más tokens por palabra fuera del inglés → peor rendimiento y mayor costo.
- **Espacios:** un espacio final puede alterar la tokenización.
- **Tokens raros** (SolidGoldMagikarp): token visto por el tokenizer pero casi no entrenado por el modelo → comportamiento errático.
- **Formato:** JSON vs YAML consume distinta cantidad de tokens.

3.2 De texto a bytes

- **Unicode:** asigna a cada carácter un **code point** (~ 150.000 definidos).
- Usar code points como vocabulario: $V \approx 150,000$, inestable, matrices enormes.
- **UTF-8:** reescribe cada code point como 1 a 4 **bytes** (0–255). Alfabeto fijo de 256, universal, sin fuera-de-vocabulario.
- ASCII: code point $\leq 127 \rightarrow 1$ byte (coincide con el número). Texto en inglés usa menos bytes.
- Límite del vocabulario de 256: T crece demasiado. Costo de atención $O(T^2)$.

3.3 Byte-Pair Encoding (BPE)

- Mecanismo: sobre los bytes, repetir: fusionar el par adyacente más frecuente en un token nuevo.
- Vocabulario inicial: 256 bytes (ids 0–255). Cada merge añade un token: id 256, 257, ...
- Tamaño final: $V = 256 + \text{número de merges}$ (hiperparámetro de diseño).
- **El entrenamiento no tiene gradiente:** es puro conteo de frecuencias sobre un corpus.
- Produce dos estructuras: **merges** (par $\rightarrow$ id nuevo, en orden) y **vocab** (id $\rightarrow$ bytes).

3.4 Encode y Decode

- **train**: descubre las reglas (por frecuencia). Se ejecuta una vez, offline.
- **encode(texto → ids)**: aplica los merges por **prioridad de merge** (orden en que se aprendieron), **no por frecuencia**. Texto → bytes → repetir: buscar el par presente cuyo merge se aprendió primero → fusionar.
- **decode(ids → texto)**: lookup en vocab. id → vocab[id] (bytes) → concatenar → decodificar UTF-8.
- **Round-trip exacto**: `decode(encode(texto)) == texto` para todo texto.
- Error común: encode por frecuencia en vez de por prioridad de merge.

3.5 Regex splitting (GPT-2/GPT-4)

- Sin restricciones, BPE fusiona a través de límites de palabra (`perro.`, `perro!` como tokens distintos).
- Solución: un patrón regex corta el texto en fragmentos (palabras, números, puntuación, espacios) **antes** de BPE. Los merges nunca cruzan fragmentos.
- GPT-4 vs GPT-2: contracciones case-insensitive, números en grupos de 1–3 dígitos, mejor manejo de espacios.
- Estos patrones se diseñan a mano, con sesgo hacia el inglés.

3.6 Tokens especiales

- Marcas de control fuera de BPE: `<|endoftext|>`, `<|im_start|>`, `<|im_end|>`, etc.
- Se agregan a mano con un id reservado, por encima de los bytes y merges.
- **encode** los separa antes de BPE. Riesgo de seguridad: hay que habilitarlos explícitamente (`allowed_special`).
- ChatML: `<|im_start|>user ... <|im_end|> <|im_start|>assistant.`

3.7 Tokenizers reales y algoritmos

Familia	Algoritmo	Librería
GPT (2/3/4)	BPE byte-level	tiktoken
BERT	WordPiece	BERT / HF tokenizers
T5	Unigram	SentencePiece
Llama 2 / Mistral	BPE (code points)	SentencePiece
Llama 3	BPE byte-level	estilo tiktoken

- **BPE**: bottom-up, fusiona pares más frecuentes.
- **WordPiece** (BERT): bottom-up, fusiona por máxima verosimilitud (no por frecuencia).
- **Unigram**: top-down, arranca con vocabulario grande y **poda** lo que menos aporta. Usa EM para estimar probabilidades, Viterbi para segmentar.
- **SentencePiece**: toolkit de Google. Corre BPE o Unigram sobre code points (no bytes). `byte_fallback` para OOV.

3.8 Notebook: Tokenización

Parte A — Tokenizer mínimo desde cero:

- Implementación de `get_stats` (contar pares) y `merge` (reemplazar par por id nuevo).
- 3 merges sobre `.aabdadaabac` → [258, 100, 258, 97, 99].
- Sobre texto real: 20 merges → compresión ~1.57x.
- Round-trip exacto verificado.
- Regex splitting con patrón de GPT-2: cada categoría (letras, números, puntuación) queda separada.

Parte B — Tokenizers reales:

- tiktoken: c1100k (GPT-4) vs o200k (GPT-4o). Vocabularios crecientes (50k → 100k → 200k).
- AutoTokenizer: GPT-2 (BPE, prefijo `G`), BERT (WordPiece, prefijo `##`), T5 (Unigram, símbolo `_`).
- Rarezas en vivo: `strawberry` → [st, raw, berry]; números partidos inconsistentemente; más tokens para japonés que inglés; JSON cuesta más tokens que YAML.

3.9 Notebook: “¿Qué es realmente la IA?”

Notebook introductorio usando Qwen3 8B con Ollama. Puntos clave:

- Un LLM **solo predice el próximo token**. Todo lo demás (memoria, personalidad, herramientas, búsqueda) es **software alrededor** del predictor.
- **raw=True**: apaga el chat template (el modelo recibe texto plano). **think=False**: apaga la cadena de razonamiento.
- La “conversación” es una ilusión: un único string con tokens especiales (`<|im_start|>`, `<|im_end|>`).
- **Temperatura**: controla el azar. $T = 0 \rightarrow$ determinístico; $T = 1 \rightarrow$ diverso.
- **Memoria**: cada llamada arranca de cero. Solución: reenviar TODO el historial.
- **Personalidad**: system prompt (texto al inicio del string).
- **Herramientas**: el modelo no calcula. Solución naive: router por keyword. Solución real: function calling.
- **Alucinación**: inventa con total seguridad cuando no sabe. Solución: RAG (inyectar contexto).
- **Alineamiento**: SFT (fine-tuning supervisado) + RLHF/preferencias. Convierte un simulador de texto en un asistente. Efecto colateral: adulación (sycophancy).
- Modelo base vs instruct: el base continúa texto; el instruct responde a instrucciones.

Clase 4: Embeddings

4.1 Parte 1: Word Embeddings

4.1.1. Los embeddings de un LLM se crean dentro del modelo

La tabla de embeddings (`nn.Embedding`) es un parámetro más, entrenado end-to-end con la misma loss de predecir el próximo token. Los modelos que vemos en esta clase son para entender el mecanismo.

4.1.2. Similitud coseno

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$$

- $\cos = 1$: misma dirección (0°). $\cos = 0$: ortogonales (90°). $\cos = -1$: opuestos (180°).
- Distancia coseno: $\text{dist} = 1 - \cos$ (más chica = más cerca).
- Compara **dirección, no tamaño**. Un vector el doble de largo pero misma dirección da $\cos = 1$.
- Se usa coseno (no euclidiana) porque la magnitud correlaciona con frecuencia, y queremos factorizarla afuera.

4.1.3. One-hot: identidad, cero significado

- Vector de V ceros con un único 1. Gigante, sparse.
- $\cos(\text{gato}, \text{perro}) = 0 = \cos(\text{gato}, \text{avión})$: todo ortogonal a todo.
- **Identidad clave**: $\text{one_hot}(i) \cdot C = C[i]$. Un embedding es leer una fila de una tabla.

4.1.4. Hipótesis distribucional y conteo

- “Una palabra se conoce por la compañía que mantiene” (Firth, 1957).
- **Matriz de co-ocurrencia**: $M[i, j] = \text{veces que } j \text{ cae en la ventana de } i$.
- Problema del conteo crudo: lo domina la frecuencia. Palabras comunes co-ocurren con todo.
- **PPMI** (Positive Pointwise Mutual Information): $\text{PMI}(w, c) = \log \frac{P(w, c)}{P(w) \cdot P(c)}$, recortado a 0. Saca el efecto de frecuencia.
- **SVD** para comprimir la matriz. Cada palabra pasa de $|\text{contextos}|$ dimensiones a d .
- Emerge geometría (temas se agrupan), pero es ruidosa y no escala.

4.1.5. Word2Vec: aprender prediciendo

- **Truco central**: inventar una tarea de predicción descartable. La tarea es la excusa; los vectores son el premio.
- **Skip-gram**: dada una palabra central, predecir sus vecinas dentro de una ventana. Self-supervised.
- Score: $\text{score}(\text{centro}, \text{contexto}) = \mathbf{v}_{\text{centro}} \cdot \mathbf{u}_{\text{contexto}}$.

- **Cuello de botella:** softmax sobre todo V ($O(|V|)$ por par).
- **Negative sampling:** de “elegir entre V ” a “¿real o falso?”. $k + 1$ problemas binarios con sigmoide. $O(k)$ en vez de $O(V)$.
- Resultado: analogías como aritmética: rey – hombre + mujer $\approx$ reina.
- Descubrimiento (Levy & Goldberg 2014): skip-gram con negative sampling factoriza implícitamente una matriz de PMI.

4.1.6. GloVe: contar y después aprender

- Combina lo mejor de contar (global, eficiente) y predecir (patrones ricos).
- El objeto: el **cociente** $P(k|\text{ice})/P(k|\text{steam})$ discrimina; la probabilidad sola, no.
- Modelo: $\mathbf{v}_i \cdot \mathbf{u}_k + b_i + b_k = \log X_{ik}$. Regresión ponderada sobre la matriz de conteos.
- **No es una red:** profundidad cero, sin no linealidad, sin softmax. Modelo bilineal.
- Los tres (SVD+PPMI, skip-gram, GloVe) factorizan la misma matriz de asociación.
- Las analogías no son emergentes: son la restricción de diseño.

4.1.7. Techo: static embeddings y polisemia

Un vector fijo por palabra, ciego al contexto. “Bank” tiene dos sentidos pero un solo vector. La resolución: embeddings **contextuales** (self-attention / Transformer).

Arco completo: one-hot (identidad) $\rightarrow$ static (significado, ciego al contexto) $\rightarrow$ contextual (Transformer).

4.1.8. Sesgos

Los embeddings heredan los sesgos del corpus. Ejemplo: man : doctor :: woman : nurse. Medir y mitigar estos sesgos es parte de responsable AI.

4.2 Parte 2: Sentence Embeddings

4.2.1. El problema

Las aplicaciones operan sobre **textos**, no palabras sueltas. Necesitamos un vector por oración, de largo fijo, que capture el significado completo.

4.2.2. Baseline: promediar word vectors

- $\text{emb}(\text{oración}) = \frac{1}{n} \sum_{i=1}^n \mathbf{v}_i$. Centro de masa semántico.
- Funciona sorprendentemente bien como piso. Pero tiene tres agujeros:
 1. **Orden:** “the dog bit the man” y “the man bit the dog” $\rightarrow$ mismo vector ($\cos = 1$).
 2. **Dilución:** oraciones largas promedian mucho relleno.
 3. **OOV:** palabras fuera de vocabulario se ignoran.

4.2.3. BERT: el encoder

- Encoder-only, bidireccional. Entrenado con masked language modeling.
- Produce un vector contextual por token. No genera texto.
- **BERT crudo no sirve para coseno**: nunca lo entrenaron para que cerca en significado $\Rightarrow$ cerca en coseno. Pooleado, rinde peor que promediar GloVe en STS.

4.2.4. SBERT (Sentence-BERT)

- SBERT = BERT + pooling + entrenamiento **contrastivo**.
- **Pooling**: de un vector por token a uno por oración. Mean pooling (más común) o token [CLS].
- **Entrenamiento contrastivo** (siamés): el mismo encoder embebe dos oraciones; la loss acerca las parecidas y aleja las distintas. Optimiza la geometría del coseno.
- Datos: pares etiquetados (paráfrasis, duplicados, NLI). Negativos gratis: in-batch negativos.
- Cierra los tres agujeros: self-attention lee el orden, pondera importancia, tokeniza en sub-words (sin OOV).
- Interfaz: `model.encode(una oración)` $\rightarrow$ un vector.

4.2.5. Embedders basados en decoder (GPT)

- Misma receta: vectores contextuales $\rightarrow$ poolear $\rightarrow$ fine-tuning contrastivo.
- Diferencia: se lee el **último** token (el único que vio toda la oración), no el [CLS].
- No se empieza de cero: arranca del GPT pre-entrenado. El pre-entrenamiento pone el **conocimiento**; el contrastivo pone la **métrica**.

4.2.6. Tres “embeddings” distintos

1. **De entrada**: vector fijo por token del vocabulario (lookup).
2. **Contextual por token**: salida de las capas con atención.
3. **Del texto** (el de RAG): UN solo vector para toda la query/documento.

4.2.7. Búsqueda semántica = la “R” de RAG

- Query + documentos $\rightarrow$ coseno $\rightarrow$ top- k . Eso es la recuperación de RAG.
- Match por **significado**, no por palabras. Una query puede no compartir palabras con el documento correcto y recuperarlo.
- **Invariante crítica**: query y documentos deben embeberse con el **mismo modelo**. El coseno solo tiene sentido dentro de un mismo espacio vectorial. Cambiar de embedder $\rightarrow$ re-indexar todo.
- Simétrico (comparar frases similares) vs asimétrico (query corta $\rightarrow$ pasaje largo).

4.2.8. Los tres modelos de un RAG

1. **Embedder** que indexa la base y embebe la query (mismo modelo).
2. **LLM generador** que escribe la respuesta (recibe texto, no vectores).
3. Se comunican por texto, no por vectores.

4.2.9. Qué embedder usar (2026)

- Benchmark: **MTEB**. Criterios: idioma, tamaño/latencia, licencia.
- APIs pagas: OpenAI, Cohere, Google (fuertes pero lock-in).
- Open source competitivo: Qwen3-Embedding, BGE-M3 (multilingüe).
- Livianos y locales: nomic-embed-text (137M, CPU), bge-m3.
- Regla: benchmarkear en tus propios datos.

4.2.10. Lo que falta para RAG en serio (Semana 8)

- **Chunking**: partir documentos largos en pedazos indexables.
- **Vector store**: Chroma, pgvector, Qdrant (en vez de lista de numpy).
- **Reranking**: segundo paso que afina el top- k .

4.3 Notebook: Word Embeddings (N1)

Recorrido ejecutable completo:

- §1: Coseno con números y plot (vectores 2D).
- §2: One-hot $\rightarrow$ lookup: `one_hot @ C == C[i]` verificado con `torch.allclose`.
- §3: Co-ocurrencia + PPMI + SVD desde cero sobre Reuters. Heatmaps de la matriz. Emerge geometría por temas (petróleo vs granos).
- §4: Word2Vec real con gensim sobre text8 (~ 17 M palabras). Vecinos de “king” y analogía king – man + woman $\approx$ queen.
- §5: GloVe pre-entrenado (400k palabras). Analogías limpias. Polisemia de “bank” (dominada por sentido financiero). Sesgos: doctor – man + woman $\approx$ nurse.
- §7: Skip-gram + negative sampling desde cero en PyTorch. Producto punto $\rightarrow$ logsigmoid. Con 50k tokens los vecinos son ruidosos: la geometría necesita escala.

4.4 Notebook: Sentence Embeddings (N2)

- §2: Baseline promedio. $\cos(\text{king}, \text{monarch}) = 0,929$ (alto), $\cos(\text{king}, \text{pizza}) = 0,759$ (bajo). Agujero del orden: dog/man $\rightarrow$ cos = 1,0.
- §3: SBERT (all-MiniLM-L6-v2 y multilingüe). Dog/man $\rightarrow$ cos = 0,98 (ya no idénticos).
- §4: Corpus de conceptos del curso. Heatmap de similitud: word2vec–negative sampling se iluminan juntos.
- §5: Búsqueda semántica a mano y con `util.semantic_search`. Query sin palabras en común recupera el documento correcto.

- §5: Invariante: $\cos(qA, qB) = 0.236$ con dos modelos distintos $\rightarrow$ espacios incompatibles.
- §6: RAG mínimo Ollama-nativo. Embedder: nomic-embed-text (768-dim). Generador: qwen3:8b. Sin RAG inventa fechas; con RAG responde desde la base.

Clase 5: Pretraining y Fine-tuning

5.1 Parte 1: Preentrenamiento — De la web cruda al modelo base

5.1.1. Tres etapas: masa, silueta, cara

- El preentrenamiento (unsupervised) construye una “masa” gigantesca de patrones. El SFT le da silueta. RLHF es la capa final que lo vuelve asistente.
- Casi todo el cómputo y los datos se gastan en el preentrenamiento. El ajuste posterior es, en comparación, una capa fina.

5.1.2. Los datos: curación empírica

Perplexity:

- $PPL = \exp\left(-\frac{1}{N} \sum_i \log P(w_i | \text{contexto})\right) = \exp(\text{cross-entropy})$.
- Interpretación: número efectivo de opciones equiprobables entre las que el modelo elige en cada paso. $PPL = 1$ (perfecto), $PPL = |V|$ (inútil).
- **Limitación:** mide fluidez con un estilo de texto, no competencia real. No compara entre tokenizers.

Ablations:

- Experimento controlado: cambiar un solo factor (filtro, fuente, dedup) y dejar todo lo demás fijo.
- Se evalúa con tareas con respuesta conocida (accuracy), no con perplexity.
- Modelos chicos y baratos, confiando en que el ranking relativo se mantiene a escala.
- **Regla:** cada decisión de curación se gana con un experimento, no con intuición.

Pipeline de curación (embudo de 8 pasos):

1. **URL:** lista negra de dominios (porno, spam, malware).
2. **Extracción de texto:** quitar HTML, menús, ads. Solo el artículo.
3. **Idioma:** detector + ruteo por idioma. Upsampling de lenguas con poca data.
4. **Gopher:** heurísticas de calidad (textos cortos, exceso de símbolos, repetición).
5. **MinHash:** deduplicación de casi-copias. Deduplicar de más puede ser contraproducente.
6. **Filtros C4:** sin puntuación, lorem ipsum, boilerplate.
7. **Filtros propios:** reglas ad-hoc para basura específica del corpus.
8. **PII:** reemplazar emails, IPs, teléfonos por placeholders.

Referencia abierta: FineWeb / FineWeb-2 (HuggingFace). Cada compañía tiene su “libro de recetas” cerrado, pero el esquema es común.

Lección central: calidad $>$ cantidad. Filtrar mejor rinde más que agregar más data cruda.

5.1.3. La escala: Scaling Laws

- Tres perillas atadas: $C \approx 6 \cdot N \cdot D$ (cómputo $\approx 6 \times$ parámetros $\times$ tokens).
- A C fijo, N y D se compensan: más parámetros $\Rightarrow$ menos tokens, y viceversa.
- **Power law**: la loss baja de forma predecible (recta en log-log) al escalar N , D o C , hasta una *irreducible loss* (entropía intrínseca del lenguaje).
- **Predicción del run**: entrenar modelos chicos, fitear la curva, extrapolar. GPT-4 predijo su loss final desde runs 1000 $\times$ más chicos.
- **Chinchilla** (DeepMind, 2022): para entrenamiento compute-óptimo, ~ 20 tokens por parámetro. Chinchilla (70B, 1.4T tokens) le ganó a Gopher (280B, 300B tokens) con el mismo cómputo.
- **Giro moderno**: sobre-entrenar por inferencia. Llama 3 8B: ~ 1900 tokens/parámetro. Modelo chico, caro una vez, barato de servir.

5.1.4. El modelo base

- Producto del pretraining: un “simulador de documentos de internet”. Competencia amplia, comportamiento crudo.
- **No se comporta como asistente**: ante “¿Cuál es la capital de Francia?” puede continuar con más preguntas (completa el patrón en vez de responder).
- El conocimiento ya está adentro: con few-shot, el base responde bien. La capacidad estaba; falta activarla.
- **Competencia** (lo que sabe) vs **comportamiento** (cómo se presenta): dos cosas distintas. Pretraining compra competencia (caro, una vez). Comportamiento es una capa final barata.

5.1.5. Continued pretraining vs fine-tuning

	Continued PT	Fine-tuning
Para qué	Conocimiento (dominio/idioma)	Comportamiento (responder, formato)
Objetivo	Next-token (self-supervised)	Pares supervisados
Datos	Texto plano, mucho	Pares, poco
Costo	US\$ cientos de miles	Chico

Regla: conocimiento nuevo $\rightarrow$ continuá pre-entrenando; comportamiento nuevo $\rightarrow$ fine-tuning. No metés un dominio entero con un dataset chico de instrucciones.

Ejemplo: modelo base + continued pretraining sobre corpus coreano (~ 200 B tokens, $\sim$ US\$0.2M) = modelo coreano. Mezclar coreano + inglés para evitar catastrophic forgetting.

5.2 Parte 2: Fine-tuning eficiente

5.2.1. Instruction tuning (SFT)

- Entrenar sobre pares (instrucción $\rightarrow$ respuesta ideal): **demostraciones** de buen comportamiento.
- Generaliza porque la variedad de tareas (resumir, traducir, explicar, clasificar) enseña la **habilidad general** de seguir instrucciones, no un catálogo de respuestas.
- **LIMA** (Less Is More for Alignment): mil ejemplos buenos y variados le ganan a un set enorme y ruidoso. Calidad $>$ cantidad.

- **Masking del prompt:** la loss se calcula solo sobre los tokens de la respuesta. La instrucción condiciona; la respuesta enseña.
- **Chat template:** serializa roles (system/user/assistant) con tokens especiales (ChatML: `<|im_start|>`, `<|im_end|>`). Mismo template en entrenamiento e inferencia.
- SFT full fine-tuning + reward model + PPO = pipeline estándar de la industria (InstructGPT, 2022).

5.2.2. PEFT: Parameter-Efficient Fine-Tuning

- Full fine-tuning mueve todos los pesos. A escala real, no entra en una GPU: ~ 16 bytes/-parámetro (pesos + gradiente + estados de Adam). Un 7B $\rightarrow \sim 112$ GB.
- PEFT congela el base y entrena $< 1\%$ de parámetros. El conocimiento ya está; para cambiar comportamiento alcanza un ajuste chico.
- Tres estrategias:
 - **Adapters:** capas chicas intercaladas en el Transformer (cuello de botella $d \rightarrow r \rightarrow d$). Suman latencia.
 - **Soft prompts / Prefix tuning:** vectores aprendidos en la entrada (o en todas las capas de atención). Gastan contexto.
 - **LoRA:** ajuste de bajo rango sobre las matrices existentes. Se fusiona $\rightarrow$ cero overhead en inferencia.

5.2.3. LoRA (Low-Rank Adaptation)

- Hallazgo empírico: el ΔW del fine-tuning tiene **rango bajo**.
- En vez de aprender $\Delta W \in \mathbb{R}^{d \times d}$, aprender $B \cdot A$ con $A \in \mathbb{R}^{r \times d}$, $B \in \mathbb{R}^{d \times r}$, $r \ll d$.
- Forward: $h = Wx + \frac{\alpha}{r} B A x$. W congelado, solo A y B entrenables.
- Inicialización: A al azar, B en ceros $\Rightarrow \Delta W = 0$ al inicio (modelo arranca idéntico al base).
- **La cuenta:** una capa $4096 \times 4096 = 16.7$ M parámetros. Con LoRA $r = 8$: $2 \times 8 \times 4096 = 65,5$ K $\approx 0,4\%$.
- Hiperparámetros: r (8–16), α (16–32), target modules (q, k, v, o de atención).
- Beneficios: memoria mínima para entrenar + adapters livianos ($\sim$ MB) e intercambiables.

5.2.4. QLoRA

- LoRA resolvió lo entrenable, pero el base congelado sigue ocupando memoria (~ 14 GB en fp16 para un 7B).
- QLoRA = LoRA + base cuantizado a **4 bits**. Almacenamiento a 4 bits, cómputo de-cuantizado al vuelo a 16 bits.
- **NF4** (4-bit NormalFloat): 16 niveles en los cuantiles de una normal. Más valores donde hay más pesos $\rightarrow$ menos error de redondeo.
- **Double quantization:** cuantiza también las constantes de escala.
- **Paged optimizers:** ante picos de memoria, descarga estados del optimizador a CPU RAM.

- Resultado: fine-tuning de un 65B en una sola GPU de 48 GB. Un 7B entra en una T4 de 16 GB (Colab gratis).
- Casi sin costo de calidad: 4-bit $\approx$ 16-bit en práctica.

5.2.5. Síntesis

- **Una sola loss** (next-token) de punta a punta: preentrenamiento $\rightarrow$ SFT $\rightarrow$ LoRA $\rightarrow$ QLoRA.
- Los datos cambiaron una vez: de texto crudo a demostraciones instrucción $\rightarrow$ respuesta.
- En cada escalón se mueven menos pesos: full FT (100 %) $\rightarrow$ LoRA ($< 1\%$) $\rightarrow$ QLoRA ($< 1\%$ + base 4-bit).
- **Techo del SFT**: clona demostraciones, llega tan lejos como sus ejemplos. No captura preferencias. Siguiendo paso: RLHF / DPO.

5.3 Notebook: Fine-tuning eficiente

Arco completo sobre SmolLM2-1.7B (base) con T4 gratis:

1. **Cargar base en 4-bit**: BitsAndBytesConfig con NF4 + double quant. 1.71B pesos, ~ 1.0 GB (vs 3.42 GB en fp16).
2. **Base no se comporta**: ante “Explain photosynthesis” genera más preguntas en loop; ante “Capital of France” responde pero no frena.
3. **Chat template**: ChatML (`<|im_start|></|im_end|>`) definido manualmente (el base no lo trae, pero los tokens ya existen en el vocabulario).
4. **Datos**: Dolly 15k, subset de 1000. 8 categorías (open_qa, classification, summarization...). Formateados con chat template.
5. **LoRA a mano**: LoRALayer (18 líneas): A al azar, B en ceros. LinearWithLoRA: $Wx + (\alpha/r)BAx$. Verificación: salida idéntica al inicio ($\Delta W = 0$).
6. **Con peft**: LoraConfig(r=8, alpha=16, targets=[q,k,v,o]). Trainable: 3.1M / 1.71B = **0.18 %**. Adapters en fp32, base en 4-bit.
7. **Entrenar**: 200 pasos con SFTTrainer, LR 2×10^{-4} . Loss baja de ~ 3.5 a ~ 2.3 .
8. **Antes/después**: ahora responde (“Paris”) y frena. Bugs críticos: model.eval() (apagar dropout de LoRA) y eos_token_id (frenar en `<|im_end|>`).
9. **Adapter guardado**: 6.1 MB (vs 3.42 GB del modelo completo).

Clase 6: Alineamiento y Evaluación

6.1 Parte 1: Alineamiento

6.1.1. La analogía de Karpathy

Tres formas de estudiar un libro = tres etapas de entrenamiento:

1. **Pretraining** = exposición, conocimiento de fondo.
2. **SFT** = problemas resueltos, imitación.
3. **RL** = problemas de práctica, probar y corregir.

6.1.2. Dominios verificables: el loop de RL

- Donde hay respuesta conocida (matemática, código), el modelo genera muchas soluciones, un **verificador** filtra las correctas, y se refuerzan.
- Una estrategia confiable (llega más seguido a la respuesta) se refuerza más, sin que nadie la etiquete.
- **El razonamiento emerge**: al optimizar por acertar, las respuestas se alargan solas (el modelo descubre que “pensar en voz alta” ayuda). Es lo que produce R1, o1, Qwen3 think.
- **Formato <think>**: el contenido del razonamiento emerge del RL; el tag <think> se impone (template o cold-start SFT).
- RL supera al que imita: como solo importa llegar a la respuesta, el modelo encuentra caminos que ningún humano usaría (AlphaGo, “movimiento 37”).

6.1.3. Dominios no verificables: preferencias

- “Un chiste sobre pelícanos”, “resumí bien esto”: no hay respuesta correcta contra qué chequear.
- **Comparar, no puntuar**: mostrar al humano varias respuestas y pedir que las ordene. Ordenar es más fácil que puntuar. La unidad mínima es un par (chosen, rejected).

6.1.4. Reward Model

- Un juez aprendido que reemplaza al verificador. Recibe (prompt, respuesta) y devuelve $r \in \mathbb{R}$.
- Nace de un GPT con la cabeza cambiada: mismas capas de Transformer, distinta cabeza (escalar en vez de distribución sobre vocabulario).
- Se entrena con Bradley-Terry: $\min - \log \sigma(r(\text{chosen}) - r(\text{rejected}))$.
- Un ranking de K respuestas se desarma en $\binom{K}{2}$ pares.

6.1.5. RLHF

- “Política” = el LLM que entrenás. La política genera, el reward model puntúa, se empuja hacia puntajes altos.
- **Reward hacking**: el RL encuentra agujeros del reward model (entradas sin sentido con puntaje altísimo, e.g. “the the the” $\rightarrow 1.0$).
- **Freno KL**: $\max \mathbb{E}[r(y)] - \beta \cdot \text{KL}(\pi \| \pi_{\text{ref}})$. Se congela una copia del SFT como ancla. β = largo de la correa.
- RLHF es un ajuste fino acotado: se corta a los pocos cientos de pasos (el reward es hackeable). Pipeline caro: hasta 4 modelos simultáneos.

6.1.6. DPO (Direct Preference Optimization)

- Mismo objetivo que RLHF (subir lo preferido sin despegarse del SFT), pero **sin reward model** y **sin PPO**: una sola loss sobre los pares.
- Explota que el juez y el generador son el mismo Transformer con distinta cabeza $\rightarrow$ colapsa las dos redes en una.
- PPO y DPO están al mismo nivel: dos caminos desde los mismos pares.

6.1.7. Constitutional AI (RLAIF)

- Reemplaza al humano por principios en lenguaje natural (“constitución”) + un modelo que juzga.
- **Fase 1:** criticar y revisar. El modelo genera, se critica contra un principio, y reescribe mejor → dataset para SFT.
- **Fase 2:** preferencias por IA. Un modelo compara dos respuestas según la constitución → pares (chosen, rejected) sin humano.
- El humano subió un nivel: de etiquetar casos a escribir principios. Es la receta de Anthropic (Claude).
- Misma trampa: el juez-IA es hackeable.

6.1.8. Síntesis del alineamiento

¿Quién puntúa?	Consecuencia
Verificador (objetivo)	Corre indefinido, razonamiento emerge, supera humanos
Reward model (proxy)	Hackeable, correa KL, techo acotado

6.2 Parte 2: Evaluación

6.2.1. Benchmarks

- Batería fija de tareas con respuesta conocida → un número comparable. MMLU, HellaSwag, GSM8K, HumanEval.
- **Mecanismo:** no es generación; se rankean log-probs de las opciones. Mide reconocimiento, no producción.
- **Saturación:** los modelos de frontera superan el benchmark → la comunidad responde con sucesores más difíciles (MMLU → MMLU-Pro → GPQA Diamond → HLE).
- **Contaminación:** benchmarks online pueden haber entrado en los datos de entrenamiento. ¿Razonó o recordó?
- Miden capacidad, no preferencia. 90 en MMLU e insufrible de usar.

6.2.2. Evaluación por preferencia

- El mismo par (chosen/rejected) que entrenó al modelo, ahora como instrumento de medición.
- **LLM-as-a-judge:** un modelo fuerte elige entre dos respuestas. Coincide con humanos ~80 %.
- Sesgos del juez: posición (prefiere A o B según orden), **verbosidad** (premia la más larga), auto-preferencia (favorece su propio estilo).
- **Chatbot Arena:** batallas ciegas entre modelos, votadas por humanos, ranking Elo.
- **Unificación:** Elo = Bradley-Terry = la misma $\sigma(\text{rating}_A - \text{rating}_B)$ que entrena el reward model y que usa DPO.

6.2.3. Modos de falla

Hallucinations:

- El modelo produce afirmaciones falsas con confianza. No es bug: el pretraining premia fluidez, no verdad. RLHF puede empeorar la calibración (sycophancy).
- **Detección:** SelfCheckGPT — muestrear N veces, medir desacuerdo. Un hecho conocido coincide; una invención diverge.
- **Mitigación:** entrenar el “no sé”, tool use / web search, RAG.

Sesgos:

- Cambiar un único término (nombre, género, nacionalidad) y medir si la respuesta cambia sistemáticamente.
- A escala: StereoSet (estereotipos), ToxiGen (toxicidad implícita).

Goodhart / Reward hacking:

- “Cuando una medida se convierte en objetivo, deja de ser una buena medida.”
- Es exactamente el reward hacking del entrenamiento, ahora en evaluación. Ejemplo: optimizar para el juez LLM = hacer respuestas más largas. Win-rate sube, calidad real no.
- La única evaluación que Goodhart no rompe: la verificable (donde hay verdad chequeable).

6.2.4. Responsible AI

- Taxonomía de Weidinger et al. (2022): 21 riesgos en 6 áreas (discriminación/hate speech, information hazards, misinformation, usos maliciosos, daños de la interacción, ambientales/socioeconómicos).
- Seis elementos: fairness, inclusiveness, reliability & safety, privacy & security, transparency, accountability.
- **Guardrail:** un segundo pase que pregunta “¿esto es seguro?” antes de devolver la respuesta. Es Constitutional AI en inferencia.
- La misma constitución que en entrenamiento definía “qué es bueno” reaparece como guardrail que define “qué es seguro mostrar”.

6.2.5. Síntesis de evaluación

- Un solo objeto —la comparación de preferencias— entrenó al modelo y lo mide. Alineamiento y evaluación son dos caras de “comparar respuestas”.
- El mismo riesgo aparece en ambos: reward hacking = Goodhart.
- Tensión de frontera: a medida que los modelos superan la capacidad humana de juzgar (scalable oversight), evaluar se vuelve el problema difícil.
- **Puente a lo que sigue:** el modelo alineado y evaluado tiene una debilidad práctica: inventa. La solución: RAG (darle una fuente).

Lecturas recomendadas

- Clase 1** Goodfellow, Bengio & Courville, *Deep Learning* (cap. 2, 4, 6.5). CS231n (Stanford). Repo micrograd (A. Karpathy).
- Clase 2** Vaswani et al. (2017), *Attention Is All You Need*. Karpathy, nanoGPT. Brown et al. (2020), *Language Models are Few-Shot Learners* (GPT-3).
- Clase 3** Sennrich et al. (2015), BPE en NLP. Radford et al. (2019), GPT-2 §2.2. Video: Karpathy, *Let's build the GPT Tokenizer*. Repo minbpe.
- Clase 4** Lena Voita, *Word Embeddings*. CS224N (Stanford). Jay Alammur, *The Illustrated Word2vec*. Reimers & Gurevych (2019), *Sentence-BERT*. Leaderboard MTEB.
- Clase 5** Hu et al. (2021), *LoRA*. Dettmers et al. (2023), *QLoRA*. Zhou et al. (2023), *LIMA*. Ouyang et al. (2022), *InstructGPT*. FineWeb (HuggingFace). Chinchilla (Hoffmann et al., 2022).
- Clase 6** Karpathy, *Deep Dive into LLMs*. Ouyang et al. (2022), InstructGPT. Rafailov et al. (2023), DPO. DeepSeek-R1 (2025). Bai et al. (2022), Constitutional AI. Zheng et al. (2023), LLM-as-a-Judge. Weidinger et al. (2022), taxonomía de riesgos.